Documentação de projetos Laravel

Código compreensível, contratos verificáveis e operação reproduzível

Do nome de uma classe ao runbook de produção

Objetivos da aula

01	**Leitura rápida**	Compreender responsabilidades sem depender do autor
02	**Integração segura**	Integrar APIs e eventos com contratos claros
03	**Exemplos executáveis**	Validar exemplos com testes e automação
04	**Evolução contínua**	Manter README, OpenAPI e decisões sempre atuais

Quem consulta cada documento

Documentação útil responde perguntas reais de desenvolvimento, integração e operação.

PÚBLICOS	PERGUNTAS
novo integrante entende o fluxo	como executar localmente
mantenedor localiza a regra	onde alterar a regra
integrador confirma o contrato	qual resposta esperar
operador executa o runbook	como diagnosticar falhas

Camadas de documentação

A documentação forma um sistema: cada camada responde a uma pergunta e possui uma fonte de verdade.

Código

- nomes revelam intenção
- tipos limitam estados
- testes mostram comportamento

Contratos

- OpenAPI descreve HTTP
- eventos expõem payloads
- ADRs registram decisões

Operação

- README prepara o ambiente
- runbooks tratam incidentes
- CI detecta divergências

A melhor fonte de verdade fica próxima do comportamento que ela descreve e pode ser verificada automaticamente.

README reproduzível

O README reduz o tempo entre clonar o repositório e executar o primeiro fluxo útil.

```
git clone git@github.com:acme/clinicflow.git
cd clinicflow
cp .env.example .env
composer install
php artisan key:generate
php artisan migrate --seed
composer run dev
```

Conteúdo mínimo

- requisitos e serviços
- instalação local
- variáveis sem segredos
- testes e comandos

Mantenha comandos copiáveis, valores seguros e uma verificação clara de que o ambiente funciona.

Uma estratégia sustentável

renomear e tipar

Use nomes, tipos e responsabilidades pequenas para reduzir a necessidade de explicações paralelas.

- nomes e tipos
- PHPDoc preciso
- testes de exemplo

2 · Complete as fronteiras

Registre contratos externos, decisões arquiteturais e procedimentos que o código não consegue expressar.

- OpenAPI e eventos
- README e ADR
- runbooks e CI

A documentação nasce com a mudança, possui dono e falha no CI quando deixa de representar o sistema.

Tipos documentam estados válidos

Enums e assinaturas reduzem estados inválidos e funcionam como documentação consultável pela IDE.

```
enum OrderStatus: string
{
    case Pending = 'pending';
    case Paid = 'paid';
    case Preparing = 'preparing';
    case Ready = 'ready';
    case Delivering = 'delivering';
    case Delivered = 'delivered';
    case Cancelled = 'cancelled';
}
```

O tipo comunica

- vocabulário do domínio
- conjunto fechado de estados
- autocompletar confiável
- refatoração segura

Prefira tipos que impeçam combinações inválidas a comentários que apenas descrevem uma convenção.

Código que explica a intenção

Um caso de uso bem nomeado, com entrada tipada e retorno explícito, explica o comportamento antes de qualquer comentário.

```
final class Checkout
{
    public function execute(
        User $customer,
        CheckoutData $data,
    ): Order {
        return DB::transaction(fn () =>
            Order::createFromCart($customer, $data)
        );
    }
}
```

Sinais de intenção

- nome orientado ao negócio
- parâmetros tipados
- efeito localizado
- retorno previsível

Se o código exige um parágrafo para ser entendido, primeiro reduza a responsabilidade e revele a intenção nos nomes.

Assinaturas revelam o modelo

Tipos de retorno em relacionamentos tornam navegação, análise estática e onboarding mais confiáveis.

```
final class Order extends Model
{
    public function restaurant(): BelongsTo
    {
        return $this->belongsTo(Restaurant::class);
    }

    public function items(): HasMany
    {
        return $this->hasMany(OrderItem::class);
    }
}
```

Contrato do modelo

- retornos genéricos
- relações nomeadas
- casts explícitos
- scopes sem ambiguidade

PHPDoc completa o sistema de tipos apenas quando o PHP não consegue expressar a informação necessária.

Comentários registram decisões

Comentários úteis registram o motivo de uma decisão surpreendente, uma limitação externa ou um risco conhecido.

```
return DB::transaction(function () use ($item, $qty) {
    $stock = Stock::query()
        ->where('menu_item_id', $item->id)
        ->lockForUpdate()
        ->firstOrFail();

    throw_if($stock->available < $qty, OutOfStock::class);
    $stock->decrement('available', $qty);

    return $stock;
});
```

Comente quando

- a regra não é óbvia
- efeito localizado
- a solução é temporária
- há referência para um ADR

Evite narrar a sintaxe. Registre a decisão e aponte para o ADR que explica contexto e consequências.

Swagger e OpenAPI no Laravel

OpenAPI descreve o contrato da API. Swagger UI transforma essa especificação em documentação navegável e testável.

```
composer require dedoc/scramble

php artisan route:list --path=docs

GET /docs/api
GET /docs/api.json
```

Do código ao contrato

- rotas e controllers
- Form Requests
- API Resources
- respostas de erro

O Scramble gera OpenAPI a partir das estruturas reais do Laravel. A equipe revisa intenção, exemplos e segurança.

Endpoint documentado pelo próprio código

Form Request e API Resource fornecem dados reais para gerar parâmetros, schemas e respostas da documentação.

```
final class StoreOrderRequest extends FormRequest
{
    public function rules(): array
    {
        return [
            'items' => ['required', 'array', 'min:1'],
            'items.*.quantity' => ['required', 'integer', 'min:1'],
        ];
    }
}

return new OrderResource($order);
```

Swagger deve mostrar

- autenticação
- exemplo de request
- respostas 201 e 422
- catálogo de erros

O contrato fica próximo do comportamento. Exemplos completam o que a análise automática não consegue inferir.

Testes são documentação executável

Um teste focado mostra como usar a funcionalidade e falha quando o comportamento público muda.

```
it('creates a paid order without overselling', function () {
    PaymentGateway::fake()->approve();

    $response = $this->actingAs($customer)->postJson(
        '/api/orders',
        CheckoutPayload::valid(),
    );

    $response->assertCreated()
        ->assertJsonPath('data.status', 'paid');
    expect(Stock::first()->available)->toBe(9);
    Event::assertDispatched(OrderPaid::class);
});
```

O exemplo deve

- ser rápido e determinístico
-
- usar dados legíveis
- verificar saída e efeito

O teste protege o comportamento observável: contrato HTTP, dados persistidos e efeitos assíncronos.

Documentação verificada no CI

A documentação deve falhar no mesmo pull request em que deixa de representar o projeto.

```
- name: Validate documentation
  run: |
    composer test
    npx markdownlint-cli2 "**/*.md"
    php artisan route:list --json > routes.json
    curl -fsS http://127.0.0.1/docs/api.json > openapi.json
    git diff --exit-code routes.json openapi.json
```

Bloqueios úteis

- links quebrados
- exemplos que falham
- OpenAPI divergente
- segredos em arquivos

README, exemplos e OpenAPI mudam no mesmo pull request que altera o comportamento público.

Projeto da aula: documentação viva

Em duplas, documente um fluxo Laravel para que outra pessoa consiga instalar, integrar, testar e diagnosticar sem ajuda do autor.

```
README.md
docs/
  api/openapi.json
  adr/0001-order-creation.md
  runbooks/queue-failures.md
.github/workflows/docs.yml
tests/Feature/CreateOrderTest.php
```

Critérios

- README executável
- Swagger abre e testa
- exemplo automatizado
- CI detecta divergência

A entrega termina quando outra dupla reproduz o fluxo e explica onde cada fonte de verdade deve ser atualizada.